Name: Janhavi Mahendra Mahale
Roll No.:79
Assignment No.: 01(a)
Title: Write and execute simple Python programs that demonstrate variable assignments, basic operations, and input/output functions.

1.1 Write a Python program to calculate **Simple Interest** using the formula.

Code:

```
# Input

P = float(input("Enter principal amount: "))

R = float(input("Enter rate of interest: "))

T = float(input("Enter time in years: "))


# Process

SI = (P * R * T) / 100

print(f"Simple Interest = ₹{SI:.2f}")

# Output
```

```
Enter principal amount: 50000
Enter rate of interest: 10
Enter time in years: 4
simple Interest= ₹20000.00
```

1.2 Write a program that takes an employee's **basic salary**, **HRA (House Rent Allowance)**, and **DA (Dearness Allowance)** and calculates **Gross Salary**.

Formula:

Gross = Basic + HRA + DA

Code:

```
basic = float(input("Enter basic salary: "))

hra = float(input("Enter HRA: "))

da = float(input("Enter DA: "))


gross = basic + hra + da

print(f"Gross Salary = ₹{gross}")
```

Output

```
Enter basic salary: 60000
Enter HRA: 4000
Enter DA: 2000
gross salary=₹66000.0
```

1.3Write a program that calculate Area of a Circle.

```
#input

radius=float(input("Enter the radius of  the circle:"))

#process

area=3.14*radius**2

print(f" The area of the circle withe radius {radius} is: {area:2f}")


# Output
```

```
Enter the radius of  the circle: 10
 The area of the circle withe radius 10.0 is: 314.000000
```

Name: Janhavi Mahendra Mahale
Roll No.:79
Assignment No.: 02(a)
Title: Write Python programs that use conditional statements (if, elif, else) and loops (for, while).

2.1.1 Electricity Bill Calculator

Code:

```
units = int(input("Enter electricity units consumed: "))
if units <= 100:
    bill = units * 5
elif units <= 300:
    bill = (100 * 5) + (units - 100) * 7
else:
    bill = (100 * 5) + (200 * 7) + (units - 300) * 10
print("Total Electricity Bill: ₹", bill)
```

#Output:

```
Enter electricity units consumed: 200
Total Electricity Bill:₹ 1200
```

2.1.2 Restaurant Billing System

Code:

```
total = 0
n = int(input("Enter number of items ordered: "))

for i in range(n):
    item = input("Enter item name: ")
    price = float(input(f"Enter price of {item}: "))
    total += price

if total > 1000:
    print("You got a 10% discount!")
    total *= 0.9

print("Total Bill Amount: ₹", total)
```

#Output:

```
Enter number of items ordered:  2
Enter item name:  shev bhaji
Enter price of shev bhaji:  120
Enter item name:  panir
Enter price of panir:  200
Total Bill Amount: ₹ 320.0
```

2.1.3 Password Validation System

Code:

```
correct_password = input("Set your password: ")
attempts = 3

while attempts > 0:
    password = input("Enter password: ")
    if password == correct_password:
        print("Access Granted!")
        break
    else:
        attempts -= 1
        print(f" Wrong password! {attempts} attempts left.")

if attempts == 0:
    print("Account locked due to multiple failed attempts.")
#Output:-
```

```
Set your password:  urvashi123
Enter password:  urvashi123
Access Granted!
```

Name: Janhavi Mahendra Mahale
Roll No.:79
Assignment No.: 02(b)
Title: Create a program to calculate the factorial of a number using a loop.

2.2.1 Factorial Calculation using for loop

```
n = int(input("Enter a number to find its factorial: "))

fact = 1

if n < 0:
   print("Factorial is not defined for negative numbers.")
else:
   for i in range(1, n + 1):
      fact *= i
   print(f"The factorial of {n} is {fact}")
```

#Output:

```
Enter a number to find its factorial:  22
The factorial of 22 is 1124000727777607680000
```

2.2.2 Factorial using while loop

```
n = int(input("Enter a number: "))

fact = 1

i = 1


while i <= n:

   fact *= i

   i += 1


print(f"The factorial of {n} is {fact}")
```

#Output:

```
Enter a number:  25
The factorial of 25 is 15511210043330985984000000
```

Name: Janhavi Mahendra Mahale
Roll No.: 79
Assignment No.: 03.1 (a)
Title: Implement a Python program to demonstrate the creation, manipulation, and use of lists.

Code: Shopping Cart Example - Demonstrating List Creation, Manipulation, and Usage

```
#1. CREATE a list (empty cart)
shopping_cart = []
print("Initial Cart:", shopping_cart)

#2. ADD ITEMS to the list
shopping_cart.append("Milk")
shopping_cart.append("Bread")
shopping_cart.append("Tea")
shopping_cart.append("Coffee")

print("\nCart after adding items:", shopping_cart)

# 3. INSERT an item at a specific position
shopping_cart.insert(1, "Butter")   # Insert Butter at position 1
print("\nCart after inserting 'Butter':", shopping_cart)

# 4. REMOVE an item
shopping_cart.remove("Tea")
print("\nCart after removing 'Tea':", shopping_cart)

# 5. UPDATE an item
shopping_cart[2] = "Whole Wheat Bread"
```

```
print("\nCart after updating 'Bread' to 'Whole Wheat Bread':", shopping_cart)

# 6. CHECK if an item exists in the cart
item = "Coffee"
if item in shopping_cart:
    print(f"\n'{item}' is available in your cart.")
else:
    print(f"\n'{item}' is NOT in your cart.")

# 7. COUNT total items
print("\nTotal items in cart:", len(shopping_cart))

# 8. SORT items alphabetically
shopping_cart.sort()
print("\nCart after sorting:", shopping_cart)

# 9. LOOP through the cart
print("\nItems in your cart:")
for product in shopping_cart:
    print("-", product)

#output
Initial Cart: []

Cart after adding items: ['Milk', 'Bread', 'Tea', 'Coffee']

Cart after inserting 'Butter': ['Milk', 'Butter', 'Bread', 'Tea', 'Coffee']

Cart after removing 'Tea': ['Milk', 'Butter', 'Bread', 'Coffee']

Cart after updating 'Bread' to 'Whole Wheat Bread': ['Milk', 'Butter', 'Whole
Wheat Bread', 'Coffee']

'Coffee' is available in your cart.
```

```
Total items in cart: 4

Cart after sorting: ['Butter', 'Coffee', 'Milk', 'Whole Wheat Bread']

Items in your cart:
- Butter
- Coffee
- Milk
- Whole Wheat Bread
```

Name: Janhavi Mahendra Mahale
Roll No.: 79
Assignment No.: 03.2 (a)
Title: Implement a Python program to demonstrate the creation, manipulation, and use of tuples.

```
# 1. Create a tuple for a flight
flight1 = ("AI101", "New Delhi", "Mumbai", "10:00 AM", "12:30 PM")
flight2 = ("AI102", "Mumbai", "Bangalore", "2:00 PM", "4:30 PM")

# 2. Access flight details
print("Flight Number:", flight1[0])
print("Origin:", flight1[1])
print("Destination:", flight1[2])
print("Departure Time:", flight1[3])
print("Arrival Time:", flight1[4])

# 3. Store multiple flights in a list of tuples
flights = [flight1, flight2]

# 4. Display all flights
print("\nAvailable Flights:")
for flight in flights:
    print(f"Flight {flight[0]}: {flight[1]} → {flight[2]} ({flight[3]} - {flight[4]})")

# 5. Count flights from a specific origin
origin_search = "Mumbai"
count = sum(1 for f in flights if f[1] == origin_search)
print(f"\nNumber of flights departing from {origin_search}:", count)
```

```
# 6. Tuple unpacking
for flight in flights:
    flight_no, origin, dest, dep_time, arr_time = flight
    print(f"\nFlight {flight_no} departs from {origin} to {dest} at {dep_time} and arrives at {arr_time}.")


#output
Flight Number: AI101
Origin: New Delhi
Destination: Mumbai
Departure Time: 10:00 AM
Arrival Time: 12:30 PM

Available Flights:
Flight AI101: New Delhi → Mumbai (10:00 AM - 12:30 PM)
Flight AI102: Mumbai → Bangalore (2:00 PM - 4:30 PM)

Number of flights departing from Mumbai: 1

Flight AI101 departs from New Delhi to Mumbai at 10:00 AM and arrives at
12:30 PM.

Flight AI102 departs from Mumbai to Bangalore at 2:00 PM and arrives at 4:30
PM.
```

Name: Janhavi Mahendra Mahale
Roll No.:79
Assignment No.: 03.3 (a)
Title: Implement a Python program to demonstrate the creation, manipulation, and use of dictionaries.

```
# Real-Life Example: Library Book Information Using Dictionary

# 1. Create a dictionary
book = {
    "title": "Python Programming",
    "author": " Allen B. Downey",
    "price": 352,
    "copies": 10
}

print("Book Details:", book)

# 2. Access dictionary values
print("\nTitle:", book["title"])
print("Author:", book["author"])

# 3. Update dictionary values
book["copies"] -= 1  # One copy borrowed
print("\nUpdated Copies:", book["copies"])

# 4. Add a new key-value pair
book["publisher"] = "ABC Publications"
print("\nAfter adding publisher:", book)
```

```
# 5. Remove a key-value pair

book.pop("price")

print("\nAfter removing price:", book)


# 6. Iterate through dictionary

print("\nBook information:")

for key, value in book.items():

   print(f"{key}: {value}")
```

#output

```
Book Details: {'title': 'Python Programming', 'author': ' Allen B. Downey',
'price': 352, 'copies': 10}

Title: Python Programming
Author:  Allen B. Downey

Updated Copies: 9

After adding publisher: {'title': 'Python Programming', 'author': ' Allen B.
Downey', 'price': 352, 'copies': 9, 'publisher': 'ABC Publications'}

After removing price: {'title': 'Python Programming', 'author': ' Allen B.
Downey', 'copies': 9, 'publisher': 'ABC Publications'}

Book information:
title: Python Programming
author:  Allen B. Downey
copies: 9
publisher: ABC Publications
```

Name: Janhavi Mahendra Mahale
Roll No.:79
Assignment No.: 03.3 (a)
Title: Implement a Python program to demonstrate the creation, manipulation, and use of dictionaries.

```
# 1. Create two sets
student1 = {"Python", "Java", "Web Development"}
student2 = {"Python", "DBMS", "Web Development"}

print("Student 1 Courses:", student1)
print("Student 2 Courses:", student2)

# 2. Add a new course
student1.add("Data Science")
print("\nAfter adding a course:", student1)

# 3. Remove a course
student1.remove("Java")
print("\nAfter removing Java:", student1)

# 4. Check whether a course exists
print("\nIs Python registered?", "Python" in student1)

# 5. Union of two sets
print("\nUnion:", student1.union(student2))

# 6. Intersection of two sets
print("\nIntersection:", student1.intersection(student2))
```

```
# 7. Difference of two sets
print("\nDifference:", student1.difference(student2))

# 8. Symmetric difference of two sets
print("\nSymmetric Difference:",
    student1.symmetric_difference(student2))

# 9. Iterate through a set
print("\nCourses of Student 1:")
for course in student1:
   print(course)

# 10. Find the number of courses
print("\nTotal Courses of Student 1:", len(student1))
#output
```

```
Student 1 Courses: {'Web Development', 'Python', 'Java'}
Student 2 Courses: {'Web Development', 'Python', 'DBMS'}

After adding a course: {'Web Development', 'Data Science', 'Python', 'Java'}

After removing Java: {'Web Development', 'Data Science', 'Python'}

Is Python registered? True

Union: {'Web Development', 'Data Science', 'Python', 'DBMS'}

Intersection: {'Web Development', 'Python'}

Difference: {'Data Science'}

Symmetric Difference: {'Data Science', 'DBMS'}

Courses of Student 1:
Web Development
Data Science
Python

Total Courses of Student 1: 3
```